Context Engineering for AI Agents

Fresh **Lessons from Building Manus**

@peakji

manus

English

Context Engineering for AI Agents: Lessons from Building Manus

Saturday, July 19

Tech

https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus

The Non-consensus

	Offload	Reduce	Retrieve	Isolate	Cache
Drew's Post	Discussed	Discussed	Discussed	Discussed	Not discussed
Manus	Used	Discouraged [1]	Not discussed	Not discussed	Used
Anthropic-researcher	Used	Used	Used	Used	Not discussed
Cognition	Not discussed	Used	Not discussed	Discouraged [2]	Not discussed
LC open-deep-research	Used	Used	Used	Used	Not discussed

[1] Information loss risk w compression is a problem. Favors offloading instead.
[2] Coordination problems w multi-agent is a risk.

@rlancemartin, "Context Engineering", LangChain

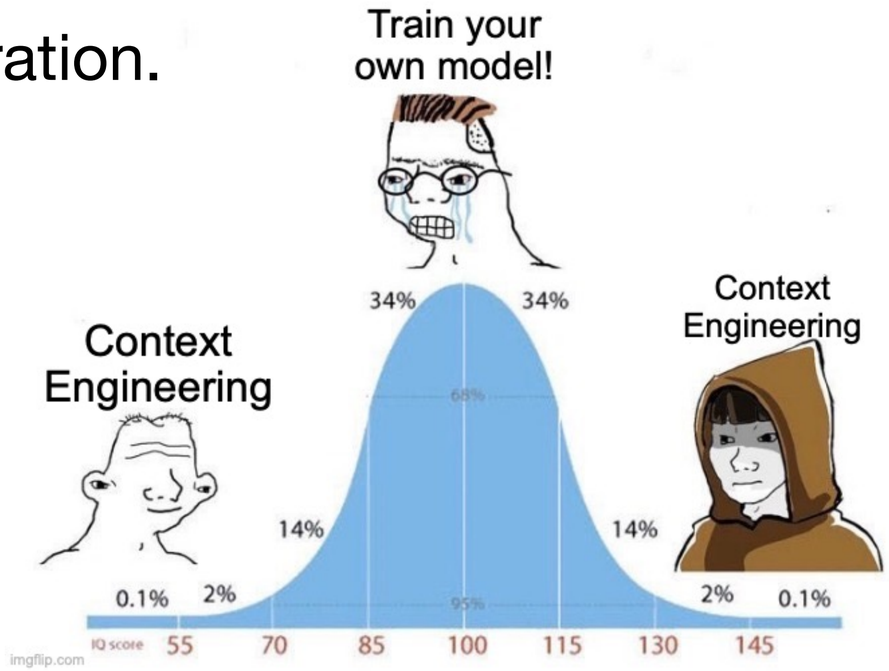
Topics

- Why Context Engineering?
- **Reduction**
  - Compaction vs. Summarization
- **Isolation**
  - “communicate by sharing memory” vs. “share memory by communicating”
- **Offloading**
  - Offload tools through a hierarchical action space

Why Context Engineering

The First Trap

- Temptation: “Why not just train my own model?”
- Fine-tuning & post-training are easier than ever.
- Reality check: model iteration limits product iteration.
- Don’t build specialized models before PMF!



Why Context Engineering

The Second Trap

- Later, when things stabilize → temptation to fine-tune with RL.
- “Let’s define an action space + reward + rollouts…”
- “… But, we need to support MCP!”
- Don’t rebuild what base-model companies already have.

Why Context Engineering

Context Engineering is the clearest and most practical boundary between application and model.

Context Reduction

Tool call

```
{
    "function": "file_write",
    "arguments": {
        "path": "/home/ubuntu/foo.txt",
        "content": "Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Morbi sed placerat
quam. Ut ultrices luctus nibh, pharetra placerat
enim tristique quis. Sed neque dolor, posuere vel
elit ac, sagittis ullamcorper odio. Pellentesque
porttitor convallis nisi, non tincidunt urna.
Quisque blandit dictum ornare..."
    }
}
```

Tool result

```
File saved to /home/ubuntu/foo.txt
```

Tool call

```
{
    "function": "browser_navigate",
    "arguments": {
        "url": "https://example.com",
        "newTab": true,
        "renderJavascript": false,
        "timeout": 30
    }
}
```

Tool result

```
Lorem ipsum dolor sit amet, consectetur adipiscing
elit. Morbi sed placerat quam. Ut ultrices luctus
nibh, pharetra placerat enim tristique quis. Sed
neque dolor, posuere vel elit ac, sagittis
ullamcorper odio. Pellentesque porttitor convallis
nisi, non tincidunt urna. Quisque blandit dictum
ornare. Praesent non ipsum a risus condimentum
fermentum ac id odio. Etiam efficitur vehicula
leo, ut vehicula nisl dapibus ac. Sed mattis, nibh
ac varius viverra, lacus risus gravida ante, id
tincidunt massa velit vulputate leo. Integer eget
vulputate augue, at volutpat turpis. Praesent
porta commodo lacus ut placerat...
```

Compaction

Tool call

```
{
    "function": "file_write",
    "arguments": {
        "path": "/home/ubuntu/foo.txt"
    }
}
```

Tool result

```
Retrieve from /home/ubuntu/foo.txt if needed
```

Tool call

```
{
    "function": "browser_navigate",
    "arguments": {
        "url": "https://example.com"
    }
}
```

Tool result

```
Revisit https://example.com if needed
```

Context Reduction

Tool call

```
{
    "function": "file_write",
    "arguments": {
        "path": "/home/ubuntu/foo.txt",
        "content": "Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Morbi sed placerat
quam. Ut ultrices luctus nibh, pharetra placerat
enim tristique quis. Sed neque dolor, posuere vel
elit ac, sagittis ullamcorper odio. Pellentesque
porttitor convallis nisi, non tincidunt urna.
Quisque blandit dictum ornare..."
    }
}
```

Tool result

```
File saved to /home/ubuntu/foo.txt
```

Tool call

```
{
    "function": "browser_navigate",
    "arguments": {
        "url": "https://example.com",
        "newTab": true,
        "renderJavascript": false,
        "timeout": 30
    }
}
```

Tool result

```
Lorem ipsum dolor sit amet, consectetur adipiscing
elit. Morbi sed placerat quam. Ut ultrices luctus
nibh, pharetra placerat enim tristique quis. Sed
neque dolor, posuere vel elit ac, sagittis
ullamcorper odio. Pellentesque porttitor convallis
nisi, non tincidunt urna. Quisque blandit dictum
ornare. Praesent non ipsum a risus condimentum
fermentum ac id odio. Etiam efficitur vehicula
leo, ut vehicula nisl dapibus ac. Sed mattis, nibh
ac varius viverra, lacus risus gravida ante, id
tincidunt massa velit vulputate leo. Integer eget
vulputate augue, at volutpat turpis. Praesent
porta commodo lacus ut placerat.
```

Summarization

Files

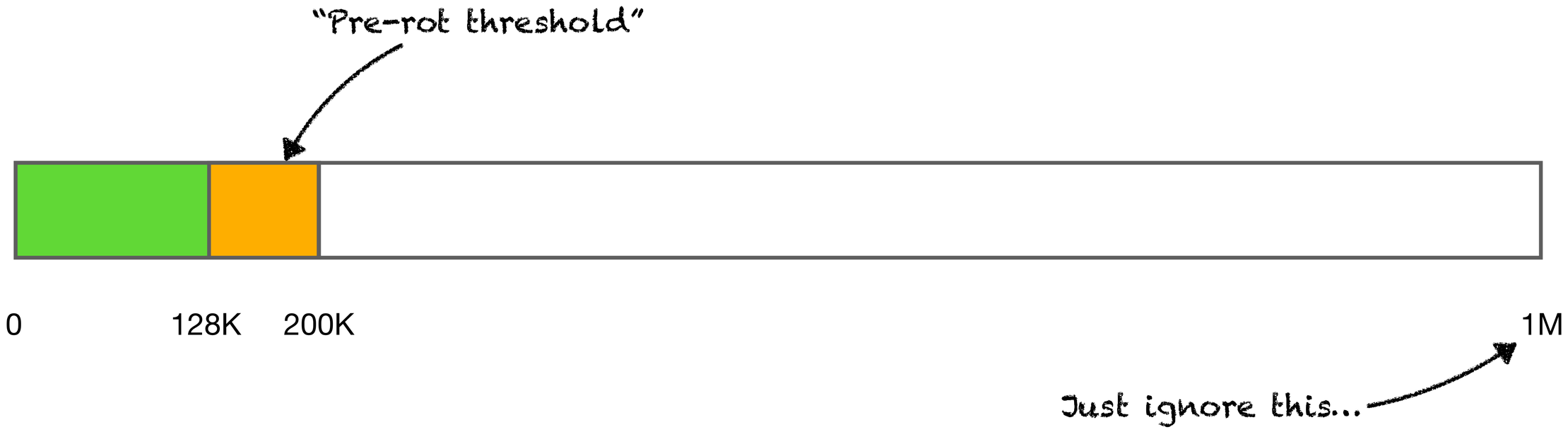
```
foo.txt
```

```
context_2025_10_13_A.log
```

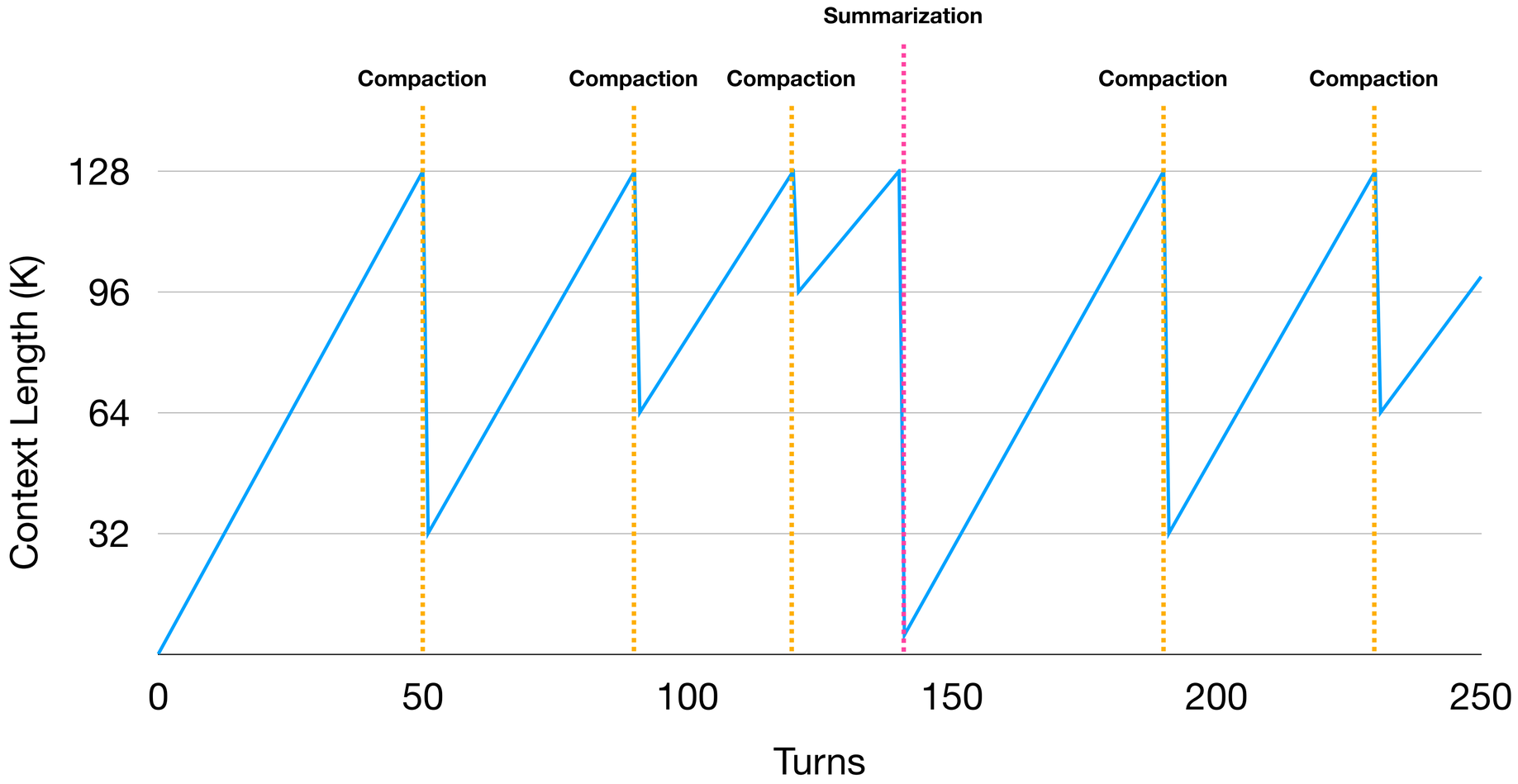
Summary

```
The user wants to … I have created 3 files in the
following paths: …
```

Context Reduction



Context Reduction



Context Isolation

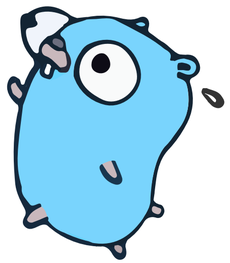
- Multi-agent setups create heavy sync overhead.
- This is not new — same in classic concurrency problems.
- We can borrow some wisdom from programming languages!

Context Isolation

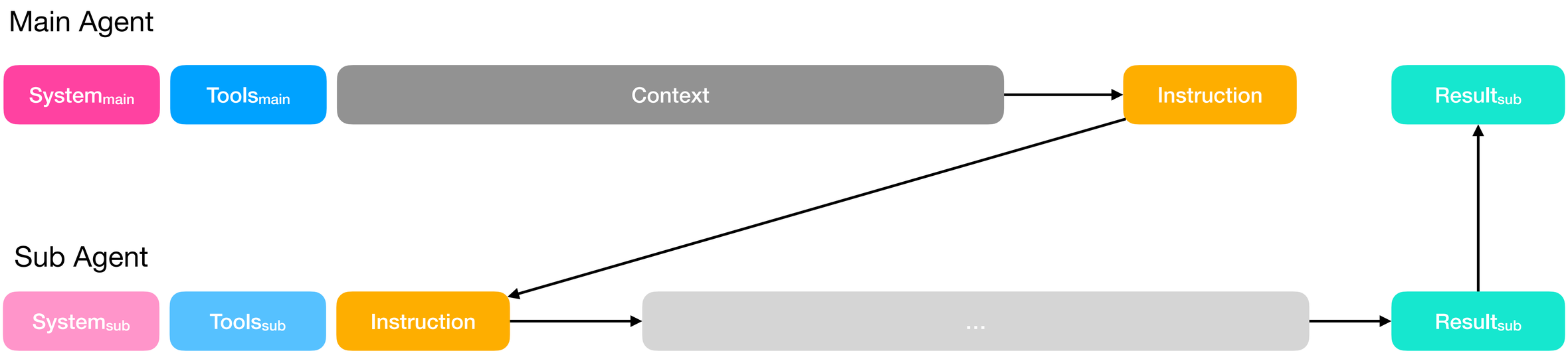
"Context"

"~~Do not communicate~~ by sharing memory;
~~instead, share memory~~ by communicating."

https://go.dev/blog/codelab-share

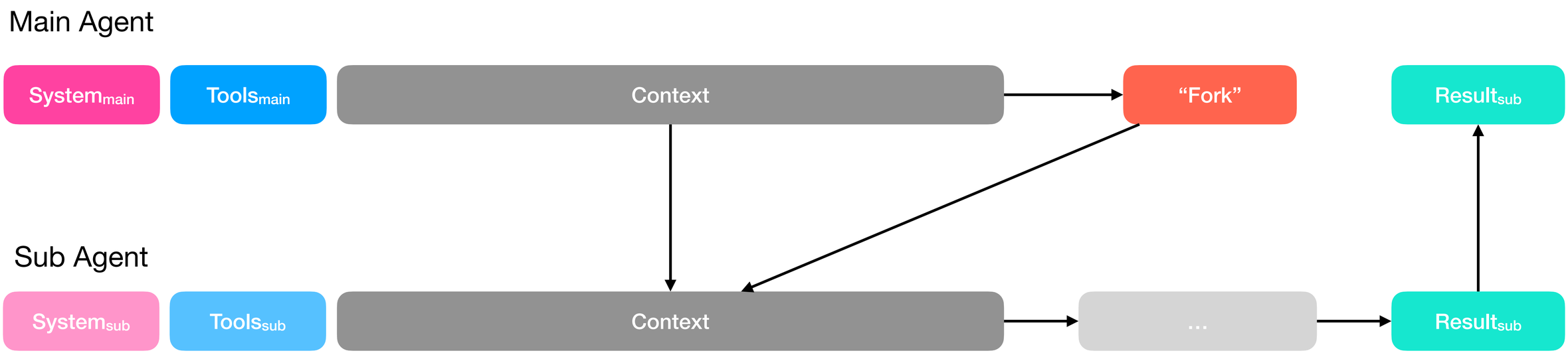


Context Isolation



By communicating

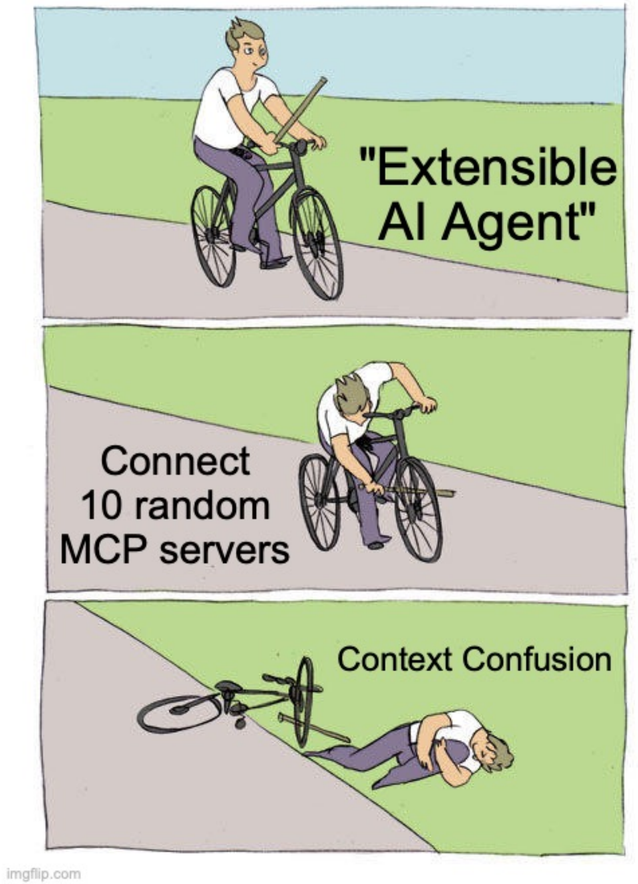
Context Isolation



By sharing context

Context Offloading

- Usually means saving working memory to files.
- But tools themselves also clutter context.
- Too many tools → confusion, invalid calls.
- So… what if we offload tools too?



Context Offloading

Hierarchical Action Space

- Three abstraction levels:
  - Function Calling
  - Sandbox Utilities
  - Packages & APIs
- Each layer balances capability, cost, and cache stability.

Context Offloading

Level 1: Function Calling

- Standard, schema-safe, but:
  - Cache breaks on every change.
  - Too many functions → context confusion.

Context Offloading

Level 2: Sandbox Utilities

- Each session runs in a full VM sandbox.
- Model can call shell utilities (CLI).
- Easy to extend without touching model context.
- Great for large outputs → write to file.

"Yes, my agent can sudo." 🤦‍💻

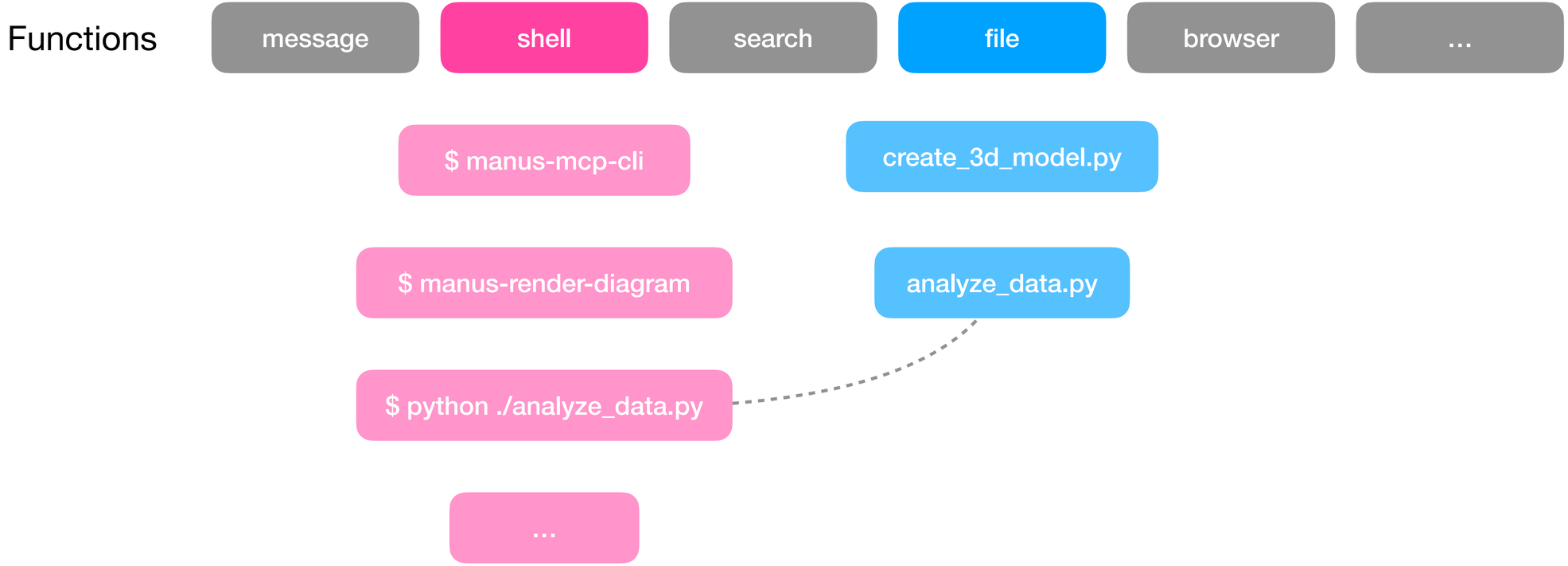
Context Offloading

Level 3: Packages & APIs

- Manus writes Python scripts to call pre-authorized APIs.
- Great for data-heavy or chained tasks.
- e.g., fetch city → get ID → get weather → summarize.
- Keep model context clean, use memory for reasoning only.

Context Offloading

- All layers still use standard function calls as proxy.
- Clean interface, cache-friendly, orthogonal design.



Bringing It All Together

- **Offload** + **Retrieve** → enable **Reduction**.
- Reliable **Retrieve** → enables **Isolation**.
- **Isolation** → reduces frequency of **Reduction**.
- All under **Cache** optimization.

Avoid Context Over-Engineering

- More context ≠ more intelligence
- Simplification beats expansion
- Our biggest gains came from removing, not adding.
- Keep the boundary clear — then get out of the model’s way!

“Build less, understand more.”

Manus AI